

Bug Bounty Report – Modifying Serialized Data Types

Vulnerability

Insecure Deserialization – Modifying Serialized Data Types

Severity

High

CVSS v3.1

8.1 (High)

CWE

- **CWE-502** – Deserialization of Untrusted Data
- **CWE-843** – Access of Resource Using Incompatible Type

OWASP Top 10

A03:2021 – Injection

Executive Summary

During this security assessment, I identified an **Insecure Deserialization** vulnerability caused by improper validation of serialized user-controlled data. By modifying the serialized data type before it was processed by the application, it was possible to influence application behavior in an unintended way.

The application trusted serialized data received from the client without performing sufficient integrity or type validation, creating an opportunity for attackers to manipulate values that should never be client-controlled.

Impact

Successful exploitation of this vulnerability could allow an attacker to:

- Manipulate serialized object data.
- Bypass application logic that depends on expected data types.
- Alter sensitive application behavior.
- Escalate privileges depending on application implementation.
- Potentially achieve unauthorized actions if insecure deserialization is used throughout the application.

The overall impact depends on how serialized objects are processed by the server, but insecure deserialization frequently leads to critical security issues.

Root Cause

The application accepted serialized data from the client and deserialized it without enforcing strict validation of expected data types or object integrity.

Missing security controls included:

- Server-side type validation
 - Integrity protection
 - Trusted serialization format enforcement
 - Secure deserialization controls
-

Risk Assessment

Category	Value
Vulnerability	Insecure Deserialization
Severity	High
Attack Complexity	Low
Privileges Required	Low
User Interaction	None
Impact	Application Logic Manipulation

Recommended Remediation

- Never trust serialized objects supplied by clients.
- Validate all serialized data before deserialization.
- Enforce strict type checking.
- Use signed or encrypted serialized objects where appropriate.
- Prefer safer serialization formats with schema validation.
- Reject unexpected object types during deserialization.
- Implement server-side integrity verification for sensitive values.

Conclusion

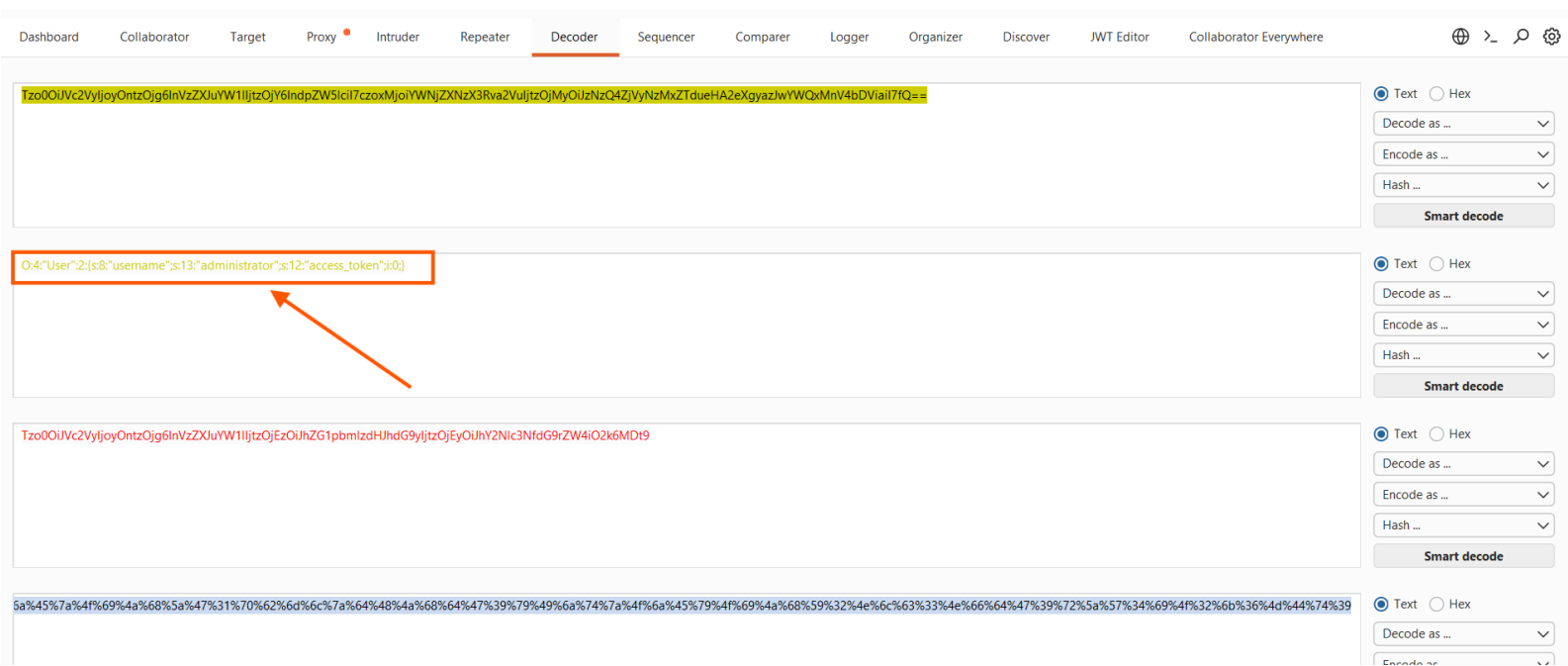
This lab demonstrates how insecure handling of serialized objects can introduce significant security risks. Proper validation, integrity protection, and secure deserialization practices are essential to prevent attackers from manipulating application logic through serialized data.

Platform: PortSwigger Web Security Academy

Category: Insecure Deserialization

Status: Lab Successfully Solved 

Purpose: Educational Security Research & Responsible Disclosure Practice



The screenshot displays the PortSwigger Web Security Academy Decoder tool. The interface features a top navigation bar with tabs: Dashboard, Collaborator, Target, Proxy, Intruder, Repeater, Decoder (active), Sequencer, Comparer, Logger, Organizer, Discover, JWT Editor, and Collaborator Everywhere. The main area shows a list of decoded payloads. The second payload, 'O:4:"User":2:{s:8:"username";s:13:"administrator";s:12:"access_token";i:0}', is highlighted with an orange box and an orange arrow pointing to it. The interface also includes a right-hand panel with options to decode or encode as Text or Hex, and a Smart decode button.